

# Phase 6: Reversing, Shifting, and Rotating in Array ADT

## 1. Big Idea of Phase 6

Phase 6 teaches how to **change the order of existing elements** inside an Array ADT.

In earlier phases, you learned how to:

1. Store array data using `A`, `size`, and `length`.
2. Add values using append and insert.
3. Remove values using delete.
4. Search values using linear search and binary search.
5. Read, update, and calculate values using get, set, max, min, sum, and average.

Now Phase 6 focuses on operations that **rearrange** the array.

The main operations are:

| Operation | Meaning                                                                              |
|-----------|--------------------------------------------------------------------------------------|
| Reverse   | Change the order from front-to-back into back-to-front                               |
| Shift     | Move elements left or right, losing one value                                        |
| Rotate    | Move elements left or right, but keep the lost value by placing it on the other side |

Simple idea:

Phase 6 = change the position of elements without changing how many valid elements exist.

## 2. Quick Review: Array ADT Structure

We usually use this fixed-size Array ADT structure:

```
struct Array {  
    int A[20];  
    int size;
```

```
int length;  
};
```

Meaning:

| Part   | Meaning                                   |
|--------|-------------------------------------------|
| A      | The actual array storage                  |
| size   | Total capacity of the array               |
| length | Number of valid elements currently stored |

Example:

```
struct Array arr = {{2, 3, 4, 5, 6}, 20, 5};
```

This means:

```
A      = [2, 3, 4, 5, 6, _, _, ...]  
size   = 20  
length = 5
```

Only these indexes are valid:

```
0, 1, 2, 3, 4
```

The last valid index is:

```
length - 1
```

So here:

```
length - 1 = 5 - 1 = 4
```

Important rule:

```
Use length when rearranging valid elements.  
Do not use size to process the real array data.
```

### 3. Important Phase 6 Rule: Length Does Not Change

Reverse, shift, and rotation change the **positions** of values.

They do not add new valid elements. They do not delete valid elements from the logical array.

Example:

Before reverse:

[2, 3, 4, 5, 6]

length = 5

After reverse:

[6, 5, 4, 3, 2]

length = 5

The order changed, but the number of valid elements stayed the same.

For left shift, one value is lost and the last position becomes `0`, but the logical length usually remains the same in this beginner version:

Before left shift:

[6, 3, 8, 5, 9]

length = 5

After left shift:

[3, 8, 5, 9, 0]

length = 5

So Phase 6 operations usually modify `A`, but not `length`.

---

## Part A: Reverse Operations

### 4. What Reverse Means

Reverse means changing the order of the array so that:

- The first element becomes the last.
- The last element becomes the first.
- The second element becomes the second last.
- The second last element becomes the second.

Example:

```
Original:  
[2, 3, 4, 5, 6]  
  
Reversed:  
[6, 5, 4, 3, 2]
```

Reverse does not sort the array. It only flips the current order.

Example:

```
Original:  
[10, 4, 15, 2]  
  
Reversed:  
[2, 15, 4, 10]
```

The result is not necessarily sorted. It is just reversed.

---

## 5. Reverse Method 1: Using an Auxiliary Array

### 5.1 Meaning of Auxiliary Array

Auxiliary means **extra** or **helping**.

So an auxiliary array is an extra array used to help solve the problem.

In this reverse method:

1. Create another array **B**.
2. Copy elements from **A** into **B** in reverse order.
3. Copy elements from **B** back into **A**.

Simple picture:

```
A = [2, 3, 4, 5, 6]  
  
Copy backward from A into B:  
B = [6, 5, 4, 3, 2]
```

Copy B back into A:  
A = [6, 5, 4, 3, 2]

## 6. Reverse Method 1 Code: Simple Version

```
void Reverse1(struct Array *arr) {  
    int B[arr->length];  
    int i, j;  
  
    for (i = arr->length - 1, j = 0; i >= 0; i--, j++) {  
        B[j] = arr->A[i];  
    }  
  
    for (i = 0; i < arr->length; i++) {  
        arr->A[i] = B[i];  
    }  
}
```

This version uses:

```
int B[arr->length];
```

That creates a temporary array of the same length as the valid elements.

Note: Some C compilers support this style, and some may not depending on the C standard/settings. A heap version using `malloc()` is also common.

## 7. Reverse Method 1 Code: Heap Version

Another version creates the auxiliary array in heap memory:

```
#include <stdlib.h>  
  
void Reverse1(struct Array *arr) {  
    int *B;  
    int i, j;  
  
    B = (int *)malloc(arr->length * sizeof(int));  
  
    for (i = arr->length - 1, j = 0; i >= 0; i--, j++) {
```

```

        B[j] = arr->A[i];
    }

    for (i = 0; i < arr->length; i++) {
        arr->A[i] = B[i];
    }

    free(B);
}

```

Important details:

| Line                     | Meaning                                                    |
|--------------------------|------------------------------------------------------------|
| <code>int *B;</code>     | Creates a pointer for the auxiliary array                  |
| <code>malloc(...)</code> | Creates heap memory for <code>length</code> integers       |
| First loop               | Copies <code>A</code> into <code>B</code> in reverse order |
| Second loop              | Copies <code>B</code> back into <code>A</code>             |
| <code>free(B);</code>    | Releases heap memory after use                             |

## 8. Reverse Method 1 Explanation

The first loop:

```

for (i = arr->length - 1, j = 0; i >= 0; i--, j++) {
    B[j] = arr->A[i];
}

```

Meaning:

- `i` starts at the last valid index of `A`.
- `j` starts at the first index of `B`.
- Copy `A[i]` into `B[j]`.
- Move `i` backward.
- Move `j` forward.

So the loop reads `A` from right to left and fills `B` from left to right.

The second loop:

```
for (i = 0; i < arr->length; i++) {
    arr->A[i] = B[i];
}
```

Meaning:

- Copy every element from **B** back into **A**.
- Now **A** becomes reversed.

## 9. Reverse Method 1 Dry Run

Given:

```
A = [2, 3, 4, 5, 6]
length = 5
```

Initial values:

```
i = length - 1 = 4
j = 0
```

First loop:

| Step | i | j | Action      | B               |
|------|---|---|-------------|-----------------|
| 1    | 4 | 0 | B[0] = A[4] | [6]             |
| 2    | 3 | 1 | B[1] = A[3] | [6, 5]          |
| 3    | 2 | 2 | B[2] = A[2] | [6, 5, 4]       |
| 4    | 1 | 3 | B[3] = A[1] | [6, 5, 4, 3]    |
| 5    | 0 | 4 | B[4] = A[0] | [6, 5, 4, 3, 2] |

Now:

```
B = [6, 5, 4, 3, 2]
```

Second loop copies **B** back into **A**:

| Step | Action      | A               |
|------|-------------|-----------------|
| 1    | A[0] = B[0] | [6, 3, 4, 5, 6] |
| 2    | A[1] = B[1] | [6, 5, 4, 5, 6] |
| 3    | A[2] = B[2] | [6, 5, 4, 5, 6] |
| 4    | A[3] = B[3] | [6, 5, 4, 3, 6] |
| 5    | A[4] = B[4] | [6, 5, 4, 3, 2] |

Final result:

A = [6, 5, 4, 3, 2]

## 10. Reverse Method 1 Complexity

| Complexity type | Result | Reason                                                  |
|-----------------|--------|---------------------------------------------------------|
| Time            | $O(N)$ | Every element is copied into B, then copied back into A |
| Space           | $O(N)$ | Extra array B is created                                |

Strictly, this method performs about  $2N$  copying operations. But in Big O notation:

$2N$  becomes  $O(N)$

So the time complexity is:

$O(N)$

The space complexity is:

$O(N)$

because the auxiliary array grows with the number of valid elements.

## 11. Reverse Method 2: Using Swaps

Reverse method 2 does not create another array.



Instead, it swaps values inside the original array.

This is called an **in-place** method.

In-place means:

The operation happens inside the same array, without using another full array.

---

## 12. Reverse Method 2 Main Idea

Use two indexes:

```
i = front index  
j = back index
```

At the start:

```
i = 0  
j = length - 1
```

Then:

1. Swap `A[i]` and `A[j]`.
2. Move `i` one step right.
3. Move `j` one step left.
4. Continue while `i < j`.

Example:

```
A = [2, 3, 4, 5, 6]
```

```
Swap A[0] and A[4]:  
[6, 3, 4, 5, 2]
```

```
Swap A[1] and A[3]:  
[6, 5, 4, 3, 2]
```

```
Stop when i and j meet.
```

## 13. Swap Helper Function

Before writing reverse using swaps, we need a swap function.

```
void swap(int *x, int *y) {  
    int temp = *x;  
    *x = *y;  
    *y = temp;  
}
```

This function receives addresses.

That is why the parameters are:

```
int *x  
int *y
```

The function swaps the actual values stored at those addresses.

---

## 14. Why Swap Needs `temp`

Suppose we want to swap two values:

```
x = 2  
y = 6
```

Wrong idea:

```
*x = *y;  
*y = *x;
```

After the first line:

```
x = 6
```

The original value `2` is lost.

So we use `temp`:

```
int temp = *x;  
*x = *y;  
*y = temp;
```

Step-by-step:

| Step | Action                       | Result   |
|------|------------------------------|----------|
| 1    | Save first value             | temp = 2 |
| 2    | Copy second into first       | x = 6    |
| 3    | Copy saved value into second | y = 2    |

Final result:

```
x = 6  
y = 2
```

Simple memory:

temp protects the first value from being lost.

---

## 15. Reverse Method 2 Code

```
void Reverse2(struct Array *arr) {  
    int i, j;  
  
    for (i = 0, j = arr->length - 1; i < j; i++, j--) {  
        swap(&arr->A[i], &arr->A[j]);  
    }  
}
```

---

## 16. Reverse Method 2 Explanation

Code:

```
for (i = 0, j = arr->length - 1; i < j; i++, j--)
```

Meaning:

| Part                                | Meaning                                         |
|-------------------------------------|-------------------------------------------------|
| <code>i = 0</code>                  | Start from the first valid element              |
| <code>j = arr-&gt;length - 1</code> | Start from the last valid element               |
| <code>i &lt; j</code>               | Continue while front index is before back index |
| <code>i++</code>                    | Move front index right                          |
| <code>j--</code>                    | Move back index left                            |

Inside the loop:

```
swap(&arr->A[i], &arr->A[j]);
```

Meaning:

- Send the address of `A[i]`.
- Send the address of `A[j]`.
- Swap the actual values.

The `&` symbol is needed because `swap()` expects addresses.

## 17. Reverse Method 2 Dry Run: Odd Number of Elements

Given:

```
A = [2, 3, 4, 5, 6]
length = 5
```

Initial values:

```
i = 0
j = 4
```

Trace:

| Step | <code>i</code> | <code>j</code> | Action                                       | Array                        |
|------|----------------|----------------|----------------------------------------------|------------------------------|
| 1    | 0              | 4              | Swap <code>A[0]</code> and <code>A[4]</code> | <code>[6, 3, 4, 5, 2]</code> |

| Step | i | j | Action             | Array           |
|------|---|---|--------------------|-----------------|
| 2    | 1 | 3 | Swap A[1] and A[3] | [6, 5, 4, 3, 2] |
| Stop | 2 | 2 | i < j is false     | Done            |

Final result:

[6, 5, 4, 3, 2]

The middle value 4 does not need to move because it is already in the middle.

## 18. Reverse Method 2 Dry Run: Even Number of Elements

Given:

A = [2, 3, 4, 5]  
length = 4

Initial values:

i = 0  
j = 3

Trace:

| Step | i | j | Action             | Array        |
|------|---|---|--------------------|--------------|
| 1    | 0 | 3 | Swap A[0] and A[3] | [5, 3, 4, 2] |
| 2    | 1 | 2 | Swap A[1] and A[2] | [5, 4, 3, 2] |
| Stop | 2 | 1 | i < j is false     | Done         |

Final result:

[5, 4, 3, 2]

For even length, the pointers cross. For odd length, the pointers meet. In both cases, the loop stops correctly when i < j becomes false.

## 19. Reverse Method 2 Complexity

| Complexity type | Result            | Reason                                                                                |
|-----------------|-------------------|---------------------------------------------------------------------------------------|
| Time            | <code>O(N)</code> | It swaps elements from both ends until the middle                                     |
| Space           | <code>O(1)</code> | It uses only a few variables: <code>i</code> , <code>j</code> , and <code>temp</code> |

Even though only about `N / 2` swaps happen, Big O ignores constants.

So:

`N / 2` becomes `O(N)`

This method is usually better than the auxiliary array method because it uses constant extra space.

## 20. Reverse Method Comparison

| Feature                 | Auxiliary Array Reverse                 | Swap Reverse               |
|-------------------------|-----------------------------------------|----------------------------|
| Extra array needed?     | Yes                                     | No                         |
| Changes original array? | Yes                                     | Yes                        |
| Time complexity         | <code>O(N)</code>                       | <code>O(N)</code>          |
| Space complexity        | <code>O(N)</code>                       | <code>O(1)</code>          |
| Main idea               | Copy into another array, then copy back | Swap front and back values |
| Usually preferred?      | Not usually                             | Yes                        |

Simple memory:

Auxiliary reverse = extra array.  
Swap reverse = same array.

## Part B: Shift Operations

### 21. What Shift Means

Shift means moving elements one position left or right.

Important idea:

In shifting, one element is lost.

For left shift:

- Every element moves one position left.
- The first element is lost.
- The last position is filled with 0.

For right shift:

- Every element moves one position right.
- The last element is lost.
- The first position is filled with 0.

---

## 22. Left Shift

### 22.1 What Left Shift Means

Left shift moves each element one position to the left.

Example:

Original:  
[6, 3, 8, 5, 9]  
  
After left shift:  
[3, 8, 5, 9, 0]

The first element 6 is lost. The last position becomes 0.

Simple memory:

Left shift = move left and lose the first element.

## 23. Left Shift Code

```
void leftShift(struct Array *arr) {  
    for (int i = 0; i < arr->length - 1; i++) {  
        arr->A[i] = arr->A[i + 1];  
    }  
  
    arr->A[arr->length - 1] = 0;  
}
```

## 24. Left Shift Explanation

The loop:

```
for (int i = 0; i < arr->length - 1; i++)
```

starts at index `0` and stops before the last valid index.

Why stop at `arr->length - 1`?

Because inside the loop we use:

```
arr->A[i + 1]
```

If `i` reached the last valid index, then `i + 1` would go outside the valid elements.

Inside the loop:

```
arr->A[i] = arr->A[i + 1];
```

This copies the next element into the current position.

After the loop:

```
arr->A[arr->length - 1] = 0;
```

This fills the last position with `0`.



## 25. Left Shift Dry Run

Given:

```
A = [6, 3, 8, 5, 9]
length = 5
```

Trace:

| Step  | i | Action         | Array           |
|-------|---|----------------|-----------------|
| Start | - | Original array | [6, 3, 8, 5, 9] |
| 1     | 0 | A[0] = A[1]    | [3, 3, 8, 5, 9] |
| 2     | 1 | A[1] = A[2]    | [3, 8, 8, 5, 9] |
| 3     | 2 | A[2] = A[3]    | [3, 8, 5, 5, 9] |
| 4     | 3 | A[3] = A[4]    | [3, 8, 5, 9, 9] |
| End   | - | A[4] = 0       | [3, 8, 5, 9, 0] |

Final result:

```
[3, 8, 5, 9, 0]
```

The original first value **6** is gone.

## 26. Right Shift

### 26.1 What Right Shift Means

Right shift moves each element one position to the right.

Example:

```
Original:
[6, 3, 8, 5, 9]

After right shift:
[0, 6, 3, 8, 5]
```

The last element `9` is lost. The first position becomes `0`.

Simple memory:

Right shift = move right and lose the last element.

---

## 27. Right Shift Code

```
void rightShift(struct Array *arr) {  
    for (int i = arr->length - 1; i > 0; i--) {  
        arr->A[i] = arr->A[i - 1];  
    }  
  
    arr->A[0] = 0;  
}
```

---

## 28. Right Shift Explanation

Right shift must move from right to left.

Why?

Because if we moved from left to right, we would overwrite values before copying them.

Correct direction:

right to left

The loop:

```
for (int i = arr->length - 1; i > 0; i--)
```

starts at the last valid index and moves backward.

Inside the loop:

```
arr->A[i] = arr->A[i - 1];
```

This copies the left neighbor into the current position.

After the loop:

```
arr->A[0] = 0;
```

This fills the first position with `0`.

---

## 29. Right Shift Dry Run

Given:

```
A = [6, 3, 8, 5, 9]  
length = 5
```

Trace:

| Step  | i | Action         | Array           |
|-------|---|----------------|-----------------|
| Start | - | Original array | [6, 3, 8, 5, 9] |
| 1     | 4 | A[4] = A[3]    | [6, 3, 8, 5, 5] |
| 2     | 3 | A[3] = A[2]    | [6, 3, 8, 8, 5] |
| 3     | 2 | A[2] = A[1]    | [6, 3, 3, 8, 5] |
| 4     | 1 | A[1] = A[0]    | [6, 6, 3, 8, 5] |
| End   | - | A[0] = 0       | [0, 6, 3, 8, 5] |

Final result:

```
[0, 6, 3, 8, 5]
```

The original last value `9` is gone.

---

# Part C: Rotation Operations

## 30. What Rotation Means

Rotation is similar to shifting, but the lost element is not thrown away.

Instead, it is saved and placed on the other side.

Simple idea:

```
Shift = lose one value.  
Rotate = keep all values by wrapping one around.
```

For left rotation:

```
[6, 3, 8, 5, 9] -> [3, 8, 5, 9, 6]
```

The first value **6** moves to the end.

For right rotation:

```
[6, 3, 8, 5, 9] -> [9, 6, 3, 8, 5]
```

The last value **9** moves to the front.

---

## 31. Left Rotation

### 31.1 What Left Rotation Means

Left rotation moves every element one position left, but saves the first element and places it at the end.

Example:

```
Original:  
[6, 3, 8, 5, 9]  
  
Save first value:  
6  
  
Shift left:
```

```
[3, 8, 5, 9, _]
```

Put saved value at end:

```
[3, 8, 5, 9, 6]
```

Simple memory:

Left rotation = first value goes to the end.

---

## 32. Left Rotation Code

```
void leftRotate(struct Array *arr) {  
    int first = arr->A[0];  
  
    for (int i = 0; i < arr->length - 1; i++) {  
        arr->A[i] = arr->A[i + 1];  
    }  
  
    arr->A[arr->length - 1] = first;  
}
```

---

## 33. Left Rotation Explanation

First:

```
int first = arr->A[0];
```

This saves the first value.

This is necessary because the first value will be overwritten during the shift.

Then:

```
for (int i = 0; i < arr->length - 1; i++) {  
    arr->A[i] = arr->A[i + 1];  
}
```

This shifts all elements left.

Finally:

```
arr->A[arr->length - 1] = first;
```

This puts the saved first value at the end.

---

## 34. Left Rotation Dry Run

Given:

```
A = [6, 3, 8, 5, 9]
length = 5
```

Save first value:

```
first = 6
```

Trace:

| Step  | i | Action         | Array           |
|-------|---|----------------|-----------------|
| Start | - | Original array | [6, 3, 8, 5, 9] |
| Save  | - | first = A[0]   | first = 6       |
| 1     | 0 | A[0] = A[1]    | [3, 3, 8, 5, 9] |
| 2     | 1 | A[1] = A[2]    | [3, 8, 8, 5, 9] |
| 3     | 2 | A[2] = A[3]    | [3, 8, 5, 5, 9] |
| 4     | 3 | A[3] = A[4]    | [3, 8, 5, 9, 9] |
| End   | - | A[4] = first   | [3, 8, 5, 9, 6] |

Final result:

```
[3, 8, 5, 9, 6]
```

No value is lost. The first value moved to the end.

---

## 35. Right Rotation

### 35.1 What Right Rotation Means

Right rotation moves every element one position right, but saves the last element and places it at the front.

Example:

```
Original:
[6, 3, 8, 5, 9]

Save last value:
9

Shift right:
[_, 6, 3, 8, 5]

Put saved value at front:
[9, 6, 3, 8, 5]
```

Simple memory:

```
Right rotation = last value goes to the front.
```

---

## 36. Right Rotation Code

```
void rightRotate(struct Array *arr) {
    int last = arr->A[arr->length - 1];

    for (int i = arr->length - 1; i > 0; i--) {
        arr->A[i] = arr->A[i - 1];
    }

    arr->A[0] = last;
}
```

---

## 37. Right Rotation Explanation

First:

```
int last = arr->A[arr->length - 1];
```

This saves the last value.

This is necessary because the last value will be overwritten during the right shift.

Then:

```
for (int i = arr->length - 1; i > 0; i--) {  
    arr->A[i] = arr->A[i - 1];  
}
```

This shifts elements right.

Finally:

```
arr->A[0] = last;
```

This puts the saved last value at the front.

## 38. Right Rotation Dry Run

Given:

```
A = [6, 3, 8, 5, 9]  
length = 5
```

Save last value:

```
last = 9
```

Trace:

| Step  | i | Action         | Array           |
|-------|---|----------------|-----------------|
| Start | - | Original array | [6, 3, 8, 5, 9] |
| Save  | - | last = A[4]    | last = 9        |
| 1     | 4 | A[4] = A[3]    | [6, 3, 8, 5, 5] |



| Step | i | Action      | Array           |
|------|---|-------------|-----------------|
| 2    | 3 | A[3] = A[2] | [6, 3, 8, 8, 5] |
| 3    | 2 | A[2] = A[1] | [6, 3, 3, 8, 5] |
| 4    | 1 | A[1] = A[0] | [6, 6, 3, 8, 5] |
| End  | - | A[0] = last | [9, 6, 3, 8, 5] |

Final result:

[9, 6, 3, 8, 5]

No value is lost. The last value moved to the front.

## Part D: Shift vs Rotation

### 39. Left Shift vs Left Rotation

| Operation     | What happens?                       | Result from [6, 3, 8, 5, 9] |
|---------------|-------------------------------------|-----------------------------|
| Left shift    | First value is lost, last becomes 0 | [3, 8, 5, 9, 0]             |
| Left rotation | First value moves to the end        | [3, 8, 5, 9, 6]             |

Main difference:

Left shift loses the first element.  
Left rotation saves the first element and moves it to the end.

### 40. Right Shift vs Right Rotation

| Operation      | What happens?                       | Result from [6, 3, 8, 5, 9] |
|----------------|-------------------------------------|-----------------------------|
| Right shift    | Last value is lost, first becomes 0 | [0, 6, 3, 8, 5]             |
| Right rotation | Last value moves to the front       | [9, 6, 3, 8, 5]             |

Main difference:

Right shift loses the last element.  
Right rotation saves the last element and moves it to the front.

## 41. Shift vs Rotate Summary

| Feature                      | Shift         | Rotate               |
|------------------------------|---------------|----------------------|
| Loses a value?               | Yes           | No                   |
| Uses saved value?            | No            | Yes                  |
| Fills empty position with 0? | Usually yes   | No                   |
| Keeps all original values?   | No            | Yes                  |
| Example idea                 | Move and lose | Move and wrap around |

Simple memory:

Shift = lose.  
Rotate = reuse.

## Part E: Complexity and Implementation Details

### 42. Complexity of Shift and Rotation

Shift and rotation both move elements one by one.

For an array with  $N$  valid elements:

| Operation      | Time complexity | Space complexity | Reason                              |
|----------------|-----------------|------------------|-------------------------------------|
| Left shift     | $O(N)$          | $O(1)$           | Moves elements left one by one      |
| Right shift    | $O(N)$          | $O(1)$           | Moves elements right one by one     |
| Left rotation  | $O(N)$          | $O(1)$           | Saves one value and shifts the rest |
| Right rotation | $O(N)$          | $O(1)$           | Saves one value and shifts the rest |

Why time is  $O(N)$ :

Every valid element may need to move.

Why space is  $O(1)$ :

Only a few extra variables are used, such as first, last, or i.

## 43. Full Phase 6 Complexity Summary

Let:

`N = arr.length`

| Operation                     | Time   | Space  | Why                                                         |
|-------------------------------|--------|--------|-------------------------------------------------------------|
| Reverse using auxiliary array | $O(N)$ | $O(N)$ | Uses an extra array                                         |
| Reverse using swaps           | $O(N)$ | $O(1)$ | Uses only index variables and swap temp                     |
| Left shift                    | $O(N)$ | $O(1)$ | Moves elements left                                         |
| Right shift                   | $O(N)$ | $O(1)$ | Moves elements right                                        |
| Left rotation                 | $O(N)$ | $O(1)$ | Saves first value, shifts left, places saved value at end   |
| Right rotation                | $O(N)$ | $O(1)$ | Saves last value, shifts right, places saved value at front |

Important comparison:

Both reverse methods are  $O(N)$  time.  
But swap reverse is better in space because it is  $O(1)$ , not  $O(N)$ .

## 44. Why Phase 6 Functions Use Pass-by-Address

Most Phase 6 operations modify the actual array.

They change the values inside `A`.

So they must receive a pointer:

```
void Reverse2(struct Array *arr)
void leftShift(struct Array *arr)
void leftRotate(struct Array *arr)
```

Because `arr` is a pointer, use the arrow operator:

```
arr->A[i]
arr->length
```

Do not use dot notation inside these functions:

```
arr.A[i]      // wrong when arr is a pointer
arr.length    // wrong when arr is a pointer
```

Simple memory:

```
Modify real array? Use pointer and arrow.
Only read array? Value and dot are okay.
```

---

## 45. Display Function for Testing

To test Phase 6 operations, we can use the same display function from earlier phases:

```
void display(struct Array arr) {
    for (int i = 0; i < arr.length; i++) {
        printf("%d ", arr.A[i]);
    }
    printf("\n");
}
```

Display uses pass-by-value because it only reads the array.

It loops only up to `length`, not `size`.

---

## Part F: Complete Phase 6 Program

### 46. Complete C Program

```
#include <stdio.h>
#include <stdlib.h>

struct Array {
    int A[20];
    int size;
    int length;
};

void display(struct Array arr) {
    for (int i = 0; i < arr.length; i++) {
        printf("%d ", arr.A[i]);
    }
    printf("\n");
}

void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

void Reverse1(struct Array *arr) {
    int *B;
    int i, j;

    B = (int *)malloc(arr->length * sizeof(int));

    for (i = arr->length - 1, j = 0; i >= 0; i--, j++) {
        B[j] = arr->A[i];
    }

    for (i = 0; i < arr->length; i++) {
        arr->A[i] = B[i];
    }

    free(B);
}

void Reverse2(struct Array *arr) {
    int i, j;
```

```

        for (i = 0, j = arr->length - 1; i < j; i++, j--) {
            swap(&arr->A[i], &arr->A[j]);
        }
    }

void leftShift(struct Array *arr) {
    for (int i = 0; i < arr->length - 1; i++) {
        arr->A[i] = arr->A[i + 1];
    }

    arr->A[arr->length - 1] = 0;
}

void rightShift(struct Array *arr) {
    for (int i = arr->length - 1; i > 0; i--) {
        arr->A[i] = arr->A[i - 1];
    }

    arr->A[0] = 0;
}

void leftRotate(struct Array *arr) {
    int first = arr->A[0];

    for (int i = 0; i < arr->length - 1; i++) {
        arr->A[i] = arr->A[i + 1];
    }

    arr->A[arr->length - 1] = first;
}

void rightRotate(struct Array *arr) {
    int last = arr->A[arr->length - 1];

    for (int i = arr->length - 1; i > 0; i--) {
        arr->A[i] = arr->A[i - 1];
    }

    arr->A[0] = last;
}

int main() {
    struct Array arr = {{2, 3, 4, 5, 6}, 20, 5};

    printf("Original array:\n");
    display(arr);

    Reverse2(&arr);
}

```

```
printf("After reverse using swaps:\n");
display(arr);

leftShift(&arr);
printf("After left shift:\n");
display(arr);

leftRotate(&arr);
printf("After left rotation:\n");
display(arr);

rightShift(&arr);
printf("After right shift:\n");
display(arr);

rightRotate(&arr);
printf("After right rotation:\n");
display(arr);

return 0;
}
```

---

## 47. Program Walkthrough

Starting array:

```
[2, 3, 4, 5, 6]
```

Step 1: Reverse using swaps

```
[6, 5, 4, 3, 2]
```

Step 2: Left shift

```
[5, 4, 3, 2, 0]
```

Step 3: Left rotation

```
[4, 3, 2, 0, 5]
```

Step 4: Right shift

```
[0, 4, 3, 2, 0]
```

Step 5: Right rotation

```
[0, 0, 4, 3, 2]
```

Important note:

The final result depends on the order in which you call the functions. Each operation modifies the same array.

---

## Part G: Common Beginner Mistakes

### 48. Mistake 1: Using `size` Instead of `length`

Wrong:

```
for (int i = 0; i < arr->size; i++)
```

Correct:

```
for (int i = 0; i < arr->length; i++)
```

Reason:

`size` includes unused capacity. `length` counts only valid elements.

---

### 49. Mistake 2: Thinking Reverse Changes Length

Reverse only changes order.

Before:



```
[2, 3, 4, 5, 6]
length = 5
```

After:

```
[6, 5, 4, 3, 2]
length = 5
```

The length stays the same.

---

## 50. Mistake 3: Forgetting to Save Before Rotation

Wrong idea:

```
// Shift first, then try to use A[0]
```

This is wrong for left rotation because the original `A[0]` gets overwritten.

Correct:

```
int first = arr->A[0];
```

Save the first value before shifting.

For right rotation, save the last value first:

```
int last = arr->A[arr->length - 1];
```

---

## 51. Mistake 4: Confusing Shift and Rotation

Shift loses one value.

Example:

```
Left shift:
[6, 3, 8, 5, 9] -> [3, 8, 5, 9, 0]
```

Rotation keeps all values.

Example:

```
Left rotation:  
[6, 3, 8, 5, 9] -> [3, 8, 5, 9, 6]
```

Memory trick:

```
Shift = lose.  
Rotate = reuse.
```

---

## 52. Mistake 5: Shifting Right in the Wrong Direction

For right shift, you must move from right to left.

Correct:

```
for (int i = arr->length - 1; i > 0; i--) {  
    arr->A[i] = arr->A[i - 1];  
}
```

Wrong direction:

```
for (int i = 0; i < arr->length - 1; i++) {  
    arr->A[i + 1] = arr->A[i];  
}
```

Why wrong?

It overwrites values before they are moved.

---

## 53. Mistake 6: Forgetting & When Calling swap

Wrong:

```
swap(arr->A[i], arr->A[j]);
```

Correct:

```
swap(&arr->A[i], &arr->A[j]);
```

Reason:

`swap()` needs addresses so it can modify the actual array elements.

---

## 54. Mistake 7: Using Dot Instead of Arrow

Inside pointer-based functions, wrong:

```
arr.length  
arr.A[i]
```

Correct:

```
arr->length  
arr->A[i]
```

Reason:

The function receives:

```
struct Array *arr
```

So `arr` is a pointer.

---

## 55. Mistake 8: Forgetting Empty Array Safety

The beginner examples assume:

```
length > 0
```

But if:

```
length = 0
```

then these are unsafe:

```
arr->A[0]  
arr->A[arr->length - 1]
```

Why?

Because there is no valid first or last element.

Safer real-code idea:

```
if (arr->length <= 0) {  
    return;  
}
```

For beginner learning, the examples use non-empty arrays.

---

## Part H: Beginner Memory Tricks

### 56. Memory Tricks

For reverse:

```
Reverse = first becomes last, last becomes first.
```

For auxiliary reverse:

```
Use extra array, copy backward, copy back.
```

For swap reverse:

```
Swap front and back until the middle.
```

For left shift:

Move left, lose first, put 0 at end.

For right shift:

Move right, lose last, put 0 at front.

For left rotation:

First value goes to the end.

For right rotation:

Last value goes to the front.

For shift vs rotate:

Shift = lose.  
Rotate = reuse.

For complexity:

Move many elements =  $O(N)$ .  
Use extra array =  $O(N)$  space.  
Use only variables =  $O(1)$  space.

## Part I: Operation Summary Table

### 57. Phase 6 Operation Summary

| Operation                     | Meaning                         | Changes array? | Changes length? | Main action            | Time   | Space  |
|-------------------------------|---------------------------------|----------------|-----------------|------------------------|--------|--------|
| Reverse using auxiliary array | Reverse order using extra array | Yes            | No              | Copy into B, copy back | $O(N)$ | $O(N)$ |
| Reverse using swaps           | Reverse order in same array     | Yes            | No              | Swap front and back    | $O(N)$ | $O(1)$ |

| Operation      | Meaning                                        | Changes array? | Changes length? | Main action                            | Time   | Space  |
|----------------|------------------------------------------------|----------------|-----------------|----------------------------------------|--------|--------|
| Left shift     | Move all elements left                         | Yes            | No              | Lose first, put 0 at end               | $O(N)$ | $O(1)$ |
| Right shift    | Move all elements right                        | Yes            | No              | Lose last, put 0 at front              | $O(N)$ | $O(1)$ |
| Left rotation  | Move all elements left and wrap first to end   | Yes            | No              | Save first, shift left, place at end   | $O(N)$ | $O(1)$ |
| Right rotation | Move all elements right and wrap last to front | Yes            | No              | Save last, shift right, place at front | $O(N)$ | $O(1)$ |

## Part J: Phase 6 Checkpoint Questions

### 58. Checkpoint Questions

You understand Phase 6 when you can answer these without looking:

1. What does reverse mean?
2. Does reverse change  $length$ ?
3. What is an auxiliary array?
4. How does reverse using an auxiliary array work?
5. Why does auxiliary reverse use  $O(N)$  extra space?
6. How does reverse using swaps work?
7. What do  $i$  and  $j$  represent in swap reverse?
8. Why does the swap reverse loop stop when  $i \geq j$ ?
9. Why is swap reverse  $O(1)$  space?
10. What does left shift do?
11. What happens to the first element in left shift?
12. Why does left shift put 0 at the last position?
13. What does right shift do?
14. What happens to the last element in right shift?
15. Why must right shift move from right to left?
16. What does left rotation do?
17. Why must left rotation save the first element before shifting?
18. What does right rotation do?
19. Why must right rotation save the last element before shifting?
20. What is the difference between shift and rotation?
21. Why are shift and rotation  $O(N)$ ?

22. Why do Phase 6 operations usually receive `struct Array *arr`?
23. When should you use `arr->length` instead of `arr.length`?
24. Why should Phase 6 operations use `length`, not `size`?
- 

## 59. Phase 6 Final Summary

Phase 6 teaches how to rearrange elements in an Array ADT.

The main operations are:

- `Reverse1` : reverse using an auxiliary array.
- `Reverse2` : reverse using swaps.
- `leftShift` : move values left and lose the first value.
- `rightShift` : move values right and lose the last value.
- `leftRotate` : move values left and place the first value at the end.
- `rightRotate` : move values right and place the last value at the front.

The most important ideas are:

Reverse changes order.  
Shift moves values and loses one.  
Rotate moves values and keeps all values by wrapping one around.

Complexity memory:

Auxiliary reverse:  $O(N)$  time,  $O(N)$  space.  
Swap reverse:  $O(N)$  time,  $O(1)$  space.  
Shift and rotation:  $O(N)$  time,  $O(1)$  space.

One-line final memory:

Phase 6 is about rearranging array elements: reverse flips the order, shift loses one element, and rotation wraps the moved element around to the other side.